

DỰ ÁN: SMART CONSUMER COMPLAINT & RESOLUTION ASSISTANT (SCCRA)

Nội dung

- I. TỔNG QUAN DỰ ÁN (About Project) 2
 - 1.1. Định nghĩa và Tổng quan..... 2
 - 1.2. Vấn đề Thực tế Cần Giải quyết..... 2
 - 1.3. Mục tiêu Đầu Ra..... 2
- II. KIẾN THỨC & CÔNG CỤ YÊU CẦU (Tech Stack & Prerequisites) 3
 - 2.1. Kiến thức Nền tảng Bắt buộc 3
 - 2.2. Kiến thức Nâng cao / Sẽ được Học trong Dự Án..... 3
 - 2.3. Công cụ & Công nghệ Sử dụng (Tech Stack) 3
- III. YÊU CẦU CHỨC NĂNG (Function Requirement) 5
 - 3.1. TỔNG QUAN VỀ CHỨC NĂNG 5
 - 3.2. USE-CASE DIAGRAM..... 5
 - 3.3. USE-CASE LIST 6
 - 3.4. USE-CASE DESCRIPTIONS 7
- IV. CÁCH HOẠT ĐỘNG (How It Works - System Architecture)..... 11
 - 4.1. Module Thu thập và Tiền xử lý Dữ liệu: 11
 - 4.2. Module Phân tích và Dashboard (EDA):..... 12
 - 4.3. Module Machine Learning (Predictive Modeling): 12
 - 4.4. Module LLM & RAG (Smart QA Assistant): 12
- V. CẤU TRÚC THƯ MỤC (Folder Tree)..... 13
- VI. XÂY DỰNG DỰ ÁN (Development Roadmap) 14
- VII. THÁCH THỨC & GIẢI PHÁP (Technical Challenges & Solutions) 16
- VIII. KẾ HOẠCH MỞ RỘNG (Future Roadmap & Extensions) 16
- IX. METRICS ĐÁNH GIÁ (Success Metrics)..... 17

I. TỔNG QUAN DỰ ÁN (About Project)

1.1. Định nghĩa và Tổng quan

Smart Consumer Complaint & Resolution Assistant (SCCRA) là một hệ thống AI/ML hoàn chỉnh (end-to-end) được thiết kế để tự động hóa và thông minh hóa quy trình xử lý khiếu nại của người tiêu dùng. Hệ thống kết hợp ba lớp xử lý chính: **Phân tích dữ liệu (Data Analytics)**, **Học máy dự đoán (Predictive Machine Learning)**, và **Trí tuệ nhân tạo ngôn ngữ lớn (Large Language Model AI)** để biến dữ liệu khiếu nại thô (unstructured text và structured metadata) thành những insights có thể hành động, hỗ trợ quyết định và thậm chí đề xuất giải pháp tự động.

1.2. Vấn đề Thực tế Cần Giải quyết

Các tổ chức (ngân hàng, công ty tài chính, sàn thương mại điện tử, cơ quan quản lý nhà nước) phải đối mặt với khối lượng lớn khiếu nại từ khách hàng mỗi ngày. Việc xử lý thủ công gặp nhiều thách thức:

- Quy mô:** Số lượng khiếu nại quá lớn để xử lý thủ công hiệu quả.
- Tính kịp thời:** Phân loại và ưu tiên chậm dẫn đến xử lý chậm, làm giảm sự hài lòng của khách hàng.
- Tính nhất quán:** Cách phân loại và đánh giá mức độ nghiêm trọng có thể khác nhau giữa các nhân viên.
- Khai thác insight:** Dữ liệu văn bản tự nhiên khó tổng hợp, phân tích xu hướng để cải thiện sản phẩm/dịch vụ.
- Tra cứu thông tin:** Nhân viên khó tìm kiếm nhanh các khiếu nại tương tự trong lịch sử để tham khảo cách giải quyết.

SCCRA ra đời để giải quyết tất cả các vấn đề trên thông qua một nền tảng tự động, thống nhất và minh bạch.

1.3. Mục tiêu Đầu Ra

Dự án hướng tới tạo ra một sản phẩm có thể chạy được (functional product) và một bộ tài liệu kỹ thuật đầy đủ, bao gồm:

- Ứng dụng Web Tổng hợp (Integrated Web Application):** Một ứng dụng Streamlit duy nhất cung cấp ba chức năng chính qua các tab riêng biệt.
- Các Mô hình ML Đã được Huấn luyện và Đóng gói:** File model (.pkl hoặc .joblib) và script inference cho hai tác vụ: Phân loại Sản phẩm và Dự đoán Mức độ Ưu tiên.
- Hệ thống Cơ sở dữ liệu Vector và RAG Pipeline:** Một cơ sở dữ liệu vector (ChromaDB/FAISS) đã được index và một pipeline hoàn chỉnh (sử dụng LangChain/LlamaIndex) để truy xuất và tạo câu trả lời dựa trên LLM.
- Mã nguồn có Tài liệu:** Toàn bộ mã nguồn trên GitHub với cấu trúc rõ ràng, docstring, và một README.md chi tiết hướng dẫn cài đặt, chạy và mở rộng.

5. **Báo cáo Phân tích (Optional):** Một tài liệu hoặc trang trong app trình bày các phát hiện từ EDA và đánh giá hiệu suất của các mô hình.

II. KIẾN THỨC & CÔNG CỤ YÊU CẦU (Tech Stack & Prerequisites)

2.1. Kiến thức Nền tảng Bắt buộc

- Lập trình Python:** Thành thạo cú pháp, cấu trúc dữ liệu, lập trình hướng đối tượng cơ bản.
- Thư viện Khoa học Dữ liệu:** Pandas (xử lý DataFrame), NumPy (tính toán số học), Matplotlib/Seaborn/Plotly (trực quan hóa).
- Machine Learning Cơ bản:** Hiểu các khái niệm phân loại (classification), huấn luyện/kiểm tra (train/test split), đánh giá mô hình (accuracy, precision, recall, F1-score).
- Xử lý Ngôn ngữ Tự nhiên (NLP) Cơ bản:** Tokenization, TF-IDF, Word Embeddings.
- SQL Cơ bản:** Có thể viết truy vấn SELECT, WHERE, GROUP BY để khám phá dữ liệu.
- Git & GitHub:** Quản lý phiên bản mã nguồn, làm việc với branch, commit, pull request.

2.2. Kiến thức Nâng cao / Sẽ được Học trong Dự Án

- LLM & Generative AI:** Khái niệm về Prompt Engineering, Retrieval-Augmented Generation (RAG), cách sử dụng LLM API (OpenAI, Gemini).
- Framework cho Ứng dụng AI:** LangChain hoặc LlamaIndex để xây dựng pipeline làm việc với LLM.
- Cơ sở dữ liệu Vector:** Khái niệm về embeddings, similarity search, và thư viện như ChromaDB hoặc FAISS.
- Web Framework cho Data App:** Streamlit để nhanh chóng xây dựng giao diện người dùng.
- Triển khai Mô hình (Model Deployment):** Cách đóng gói mô hình và tích hợp vào ứng dụng.

2.3. Công cụ & Công nghệ Sử dụng (Tech Stack)

Loại	Công cụ	Mục đích Sử dụng
Ngôn ngữ & Môi trường	Python 3.9+	Ngôn ngữ lập trình chính
	Jupyter Notebook	Khám phá dữ liệu, thử nghiệm mô hình
Data Processing & ML	Pandas, NumPy	Thao tác và làm sạch dữ liệu
	Scikit-learn	Xây dựng và huấn luyện mô hình ML cổ điển

Loại	Công cụ	Mục đích Sử dụng
LLM & AI	NLTK / spaCy	Xử lý văn bản tiếng Anh cơ bản
	OpenAI API (gpt-3.5-turbo) hoặc Llama 3 (qua Groq/Ollama)	LLM backend cho RAG và generation
	LangChain / LlamaIndex	Framework để xây dựng pipeline RAG và agents
	ChromaDB / FAISS	Lưu trữ và tìm kiếm vector embeddings
Visualization & App	sentence-transformers	Tạo embeddings cho văn bản (nếu dùng model open-source)
	Streamlit	Xây dựng ứng dụng web và dashboard
	Plotly, Seaborn	Tạo biểu đồ tương tác và tĩnh trong app
Database & Storage	SQLite / CSV	Lưu trữ dữ liệu gốc và đã qua xử lý
	ChromaDB (Persistent)	Lưu trữ vector database
Development & Deployment	Git, GitHub	Quản lý mã nguồn và hợp tác
	Pip / Conda	Quản lý môi trường và thư viện
	Streamlit Community Cloud	Triển khai public app (free tier)

III. YÊU CẦU CHỨC NĂNG (Function Requirement)

3.1. TỔNG QUAN VỀ CHỨC NĂNG

Hệ thống Smart Consumer Complaint & Resolution Assistant (SCCRA) là một nền tảng AI end-to-end cung cấp ba nhóm chức năng chính:

Phân tích Dữ liệu Thông minh: Dashboard tương tác thời gian thực cho phép khám phá, trực quan hóa và phân tích xu hướng khiếu nại từ nhiều chiều dữ liệu khác nhau.

Tự động hóa Xử lý bằng Machine Learning: Tích hợp các mô hình ML để tự động phân loại khiếu nại và đánh giá mức độ ưu tiên, giảm tải công việc thủ công và tăng tốc độ xử lý.

Hỗ trợ Quyết định bằng LLM: Cung cấp chatbot hỏi-đáp thông minh (RAG) cho phép người dùng tra cứu thông tin chuyên sâu từ kho dữ liệu bằng ngôn ngữ tự nhiên, hỗ trợ ra quyết định nhanh chóng và có căn cứ.

3.2. USE-CASE DIAGRAM



Lưu ý: Diagram trên thể hiện các Actor chính tương tác với hệ thống SCCRA và các hệ thống bên ngoài mà SCCRA tích hợp.

3.3. USE-CASE LIST

ID	Tên Use-Case	Tác nhân chính	Tác nhân phụ / Hệ thống	Mô tả ngắn	Độ ưu tiên
UC-DA-01	Khám phá & Trực quan hóa Dữ liệu	Business Analyst	Database, Plotly/Streamlit	Sử dụng Dashboard tương tác để lọc, xem biểu đồ, và phân tích xu hướng khiếu nại theo thời gian, sản phẩm, công ty.	Cao
UC-DA-02	Xuất báo cáo Phân tích	Business Analyst	Database, Report Generator	Xuất kết quả phân tích và biểu đồ dưới dạng file PDF hoặc Excel để báo cáo.	Trung bình
UC-ML-01	Phân loại Sản phẩm Tự động (Real-time)	Complaint Handler	ML Model Service, Vectorizer	Nhập mô tả khiếu nại mới, hệ thống tự động phân loại vào nhóm sản phẩm phù hợp và hiển thị kết quả.	Cao
UC-ML-02	Dự đoán Mức độ Ưu tiên (Real-time)	Complaint Handler	ML Model Service	Nhập mô tả khiếu nại, hệ thống dự đoán mức độ ưu tiên ("High"/"Low") để hỗ trợ ưu tiên xử lý.	Cao
UC-ML-03	Phân loại Hàng loạt (Batch Processing)	Data Scientist	Batch Job Scheduler, ML Service	Upload file CSV chứa nhiều khiếu nại, hệ thống xử lý hàng loạt và trả về file kết quả với các nhãn dự đoán.	Cao
UC-RG-01	Hỏi-Đáp Thông minh về Dữ liệu (Chatbot)	Complaint Handler, BA	RAG Pipeline, LLM API, Vector DB	Đặt câu hỏi tự nhiên về dữ liệu khiếu nại (ví dụ: "Khiếu nại phổ biến nhất quý 3?"), nhận câu trả lời tổng hợp từ LLM dựa trên dữ liệu thực.	Cao
UC-RG-02	Tìm kiếm Khiếu nại Tương tự	Complaint Handler	Vector DB, Similarity Search	Nhập một khiếu nại mẫu, hệ thống tìm và hiển thị danh sách các khiếu nại có nội dung tương tự nhất trong lịch sử để tham khảo cách xử lý.	Trung bình
UC-AD-01	Quản lý Vòng đời Mô hình ML	Data Scientist, Admin	Model Registry, Version Control	Upload model mới, so sánh hiệu suất với model hiện tại, chọn model để deploy/deprecate.	Trung bình

ID	Tên Use-Case	Tác nhân chính	Tác nhân phụ / Hệ thống	Mô tả ngắn	Độ ưu tiên
UC-AD-02	Cấu hình Hệ thống RAG & LLM	System Admin	LLM API, Vector DB Config	Thay đổi LLM model (GPT-3.5/4), điều chỉnh tham số (temperature), cấu hình prompt templates và chunking strategy.	Trung bình
UC-AD-03	Giám sát Hiệu năng & Sức khỏe Hệ thống	System Admin	Monitoring Service, Logging	Xem dashboard giám sát: latency của API, số lượng request, trạng thái kết nối đến LLM/Vector DB, lỗi hệ thống.	Thấp
UC-AD-04	Quản lý Người dùng & Phân quyền (RBAC)	System Admin	Authentication Service	Thêm/xóa người dùng, gán vai trò (Admin, Analyst, Handler) để kiểm soát quyền truy cập vào các chức năng khác nhau.	Trung bình
UC-DP-01	Đồng bộ & Làm mới Dữ liệu Tự động	Hệ thống (Scheduler)	External Data Source (CFPB API)	Tự động chạy job định kỳ để tải dataset mới nhất từ nguồn bên ngoài, làm sạch và cập nhật vào hệ thống.	Cao

3.4. USE-CASE DESCRIPTIONS

UC-ML-01: PHÂN LOẠI SẢN PHẨM TỰ ĐỘNG (REAL-TIME)

Tự động phân loại văn bản khiếu nại mới vào danh mục sản phẩm phù hợp.

Tác nhân chính: Complaint Handler (Nhân viên xử lý khiếu nại)

Tác nhân phụ: ML Model Service, Text Vectorizer

Mục tiêu: Giảm thời gian phân loại thủ công, đảm bảo tính nhất quán.

Tiền điều kiện:

- ML Model cho phân loại sản phẩm đã được huấn luyện và deploy.
- Người dùng đã đăng nhập và có quyền sử dụng tính năng ML Demo.
- Hệ thống ML Service đang hoạt động.

Dòng sự kiện chính (Basic Flow):

- Người dùng điều hướng đến tab "ML Demo" trong ứng dụng.
- Người dùng chọn chức năng "Phân loại Sản phẩm".
- Người dùng nhập hoặc dán nội dung văn bản khiếu nại vào ô textbox.
- Người dùng nhấn nút "Phân tích".

5. Ứng dụng gửi văn bản đến ML Model Service qua API.
6. ML Model Service thực hiện tiền xử lý văn bản (tokenization, cleaning).
7. ML Model Service chuyển đổi văn bản thành vector đặc trưng (sử dụng TF-IDF hoặc Embedding đã được huấn luyện).
8. Mô hình ML thực hiện dự đoán và trả về kết quả: Tên sản phẩm dự đoán và Độ tin cậy (Confidence Score từ 0-100%).
9. Ứng dụng hiển thị kết quả rõ ràng trên giao diện, ví dụ: "Sản phẩm: Thẻ tín dụng (Credit Card) - Độ tin cậy: 92%".
10. Hệ thống tự động ghi log yêu cầu và kết quả dự đoán vào database để theo dõi và đánh giá model sau này.

Dòng ngoại lệ (Alternative Flows):

A1: Văn bản đầu vào quá ngắn hoặc không rõ ràng.

Nếu văn bản nhập vào có ít hơn 10 từ, hệ thống hiển thị cảnh báo: *"Văn bản có thể quá ngắn để phân tích chính xác. Vui lòng nhập mô tả chi tiết hơn (nếu có)."*

Quá trình dự đoán vẫn tiếp tục, nhưng Confidence Score có thể được đánh dấu là thấp.

A2: Độ tin cậy của dự đoán thấp.

Nếu Confidence Score trả về < 60%, hệ thống hiển thị kết quả kèm cảnh báo nổi bật: *"Độ tin cậy thấp. Kết quả này cần được xem xét thủ công."*

Hệ thống có thể đề xuất 2-3 danh mục sản phẩm có điểm số cao nhất.

A3: ML Model Service không phản hồi.

Nếu request đến ML Service timeout (sau 10 giây), ứng dụng hiển thị thông báo lỗi: *"Dịch vụ phân tích tạm thời gián đoạn. Vui lòng thử lại sau."*

Ứng dụng có thể chuyển sang chế độ dự phòng (fallback), sử dụng một bộ rule-based classifier đơn giản được nhúng sẵn trong front-end để đưa ra dự đoán sơ bộ.

Hậu điều kiện (Post-conditions):

1. Kết quả phân loại được hiển thị cho người dùng.
2. Một bản ghi (log) chứa văn bản đầu vào, kết quả dự đoán, timestamp, và user ID được lưu vào database cho mục đích audit và retraining.

UC-RG-01: HỎI-ĐÁP THÔNG MINH VỀ DỮ LIỆU (CHATBOT)

Tra cứu thông tin từ kho dữ liệu khiêu nại bằng câu hỏi tiếng Anh tự nhiên thông qua giao diện chat.

Tác nhân chính: Complaint Handler, Business Analyst

Tác nhân phụ: RAG Pipeline, LLM API (OpenAI GPT), Vector Database (ChromaDB)

Mục tiêu: Cho phép người dùng khai thác insights từ dữ liệu một cách nhanh chóng, linh hoạt mà không cần biết SQL hoặc kỹ thuật phức tạp.

Tiền điều kiện:

1. Vector Database đã được index đầy đủ với dữ liệu khiếu nại.
2. Kết nối đến LLM API ổn định và có đủ credit/token.
3. Người dùng đã đăng nhập.

Dòng sự kiện chính (Basic Flow):

1. Người dùng điều hướng đến tab "Smart QA Assistant".
2. Giao diện hiển thị một khung chat, nơi người dùng có thể nhập câu hỏi.
3. Người dùng nhập câu hỏi bằng tiếng Anh, ví dụ: *"What were the top 3 complaint issues about Bank of America in January 2024?"*
4. Người dùng nhấn nút "Gửi" hoặc phím Enter.
5. Hệ thống RAG Pipeline xử lý:
 - a. Retrieval: Chuyển đổi câu hỏi thành embedding vector, tìm kiếm trong Vector DB để lấy ra top K đoạn văn bản (chunks) có liên quan nhất từ kho khiếu nại.
 - b. Augmentation: Kết hợp các đoạn văn bản tìm được thành một "ngữ cảnh" (context).
 - c. Generation: Tạo một prompt kỹ lưỡng gửi đến LLM API, bao gồm: *Hướng dẫn hệ thống ("Bạn là trợ lý phân tích khiếu nại...")*, *Ngữ cảnh vừa tìm được*, *Câu hỏi của người dùng*, *Yêu cầu định dạng câu trả lời và trích dẫn nguồn*.
6. LLM xử lý prompt và sinh ra câu trả lời dựa trên ngữ cảnh được cung cấp.
7. Ứng dụng hiển thị câu trả lời được định dạng đẹp trong khung chat. Câu trả lời nên bao gồm:
 - a. Phần tóm tắt/thông tin chính.
 - b. Các điểm bullet rõ ràng (nếu phù hợp).
10. Trích dẫn nguồn (Citations): Liên kết hoặc ID đến các khiếu nại gốc đã được sử dụng, cho phép người dùng click để xem chi tiết.

Dòng ngoại lệ (Alternative Flows):

A1: Không tìm thấy ngữ cảnh phù hợp.

Nếu hệ thống không thể tìm thấy bất kỳ đoạn văn bản nào có độ tương tự trên ngưỡng tối thiểu, nó sẽ trả lời: *"Xin lỗi, tôi không tìm thấy thông tin cụ thể về chủ đề này trong cơ sở dữ liệu hiện tại. Vui lòng thử câu hỏi khác hoặc làm rõ từ khóa."*

A2: Câu hỏi không liên quan đến dữ liệu khiếu nại.

Hệ thống được cấu hình để nhận diện các câu hỏi ngoài phạm vi (off-topic). Nếu phát hiện, nó sẽ trả lời: *"Tôi được thiết kế để trả lời các câu hỏi về dữ liệu khiếu nại người tiêu dùng. Tôi không thể trả lời câu hỏi về [chủ đề khác]."*

A3: LLM API gặp lỗi hoặc hết quota.

Ứng dụng hiển thị thông báo: "*Dịch vụ AI đang gặp sự cố. Vui lòng thử lại sau ít phút.*" và ghi log lỗi chi tiết.

Hậu điều kiện (Post-conditions):

1. Câu hỏi và câu trả lời được lưu vào lịch sử chat của phiên làm việc.
2. Một bản ghi tương tác (gồm câu hỏi, các chunk được truy xuất, và câu trả lời) được lưu để đánh giá chất lượng hệ thống RAG và tối ưu hóa sau này.

UC-AD-01: QUẢN LÝ VÒNG ĐỜI MÔ HÌNH ML

Upload, so sánh, và quyết định triển khai các phiên bản mô hình ML mới.

Tác nhân chính: Data Scientist, System Admin

Tác nhân phụ: Model Registry (MLflow), Version Control, Testing Pipeline

Tiền điều kiện: Người dùng có quyền Admin hoặc Data Scientist.

Dòng sự kiện chính (Basic Flow):

1. DS/Admin đăng nhập vào trang "Model Management".
2. Hệ thống liệt kê các model hiện đang được quản lý, với thông tin: Tên model, Phiên bản, Ngày upload, Hiệu suất (Accuracy, F1-score), Trạng thái (Production/Staging/Archived).
3. Người dùng nhấn nút "Upload New Model".
4. Người dùng chọn file model (.pkl/.joblib) và file metadata (chứa các metrics đánh giá trên tập test).
5. Hệ thống tự động validate model (kiểm tra định dạng, tương thích với vectorizer hiện tại).
6. Hệ thống chạy Automated Evaluation trên một tập test chuẩn (golden dataset) để tính toán các metrics khách quan.
7. Hệ thống hiển thị bảng so sánh chi tiết giữa model mới và model đang production: sự khác biệt về accuracy, precision, recall cho từng lớp.
8. Người dùng xem xét kết quả và chọn một hành động:
 - "Deploy to Production": Thay thế model cũ. Hệ thống sẽ tự động cập nhật ML Service.
 - "Save as Staging": Lưu model để thử nghiệm A/B testing.
 - "Reject": Không chấp nhận model mới.
9. Hệ thống cập nhật trạng thái model, ghi log hành động, và thông báo kết quả cho người dùng.

Dòng ngoại lệ (Alternative Flows):

A1: Model validation failed.

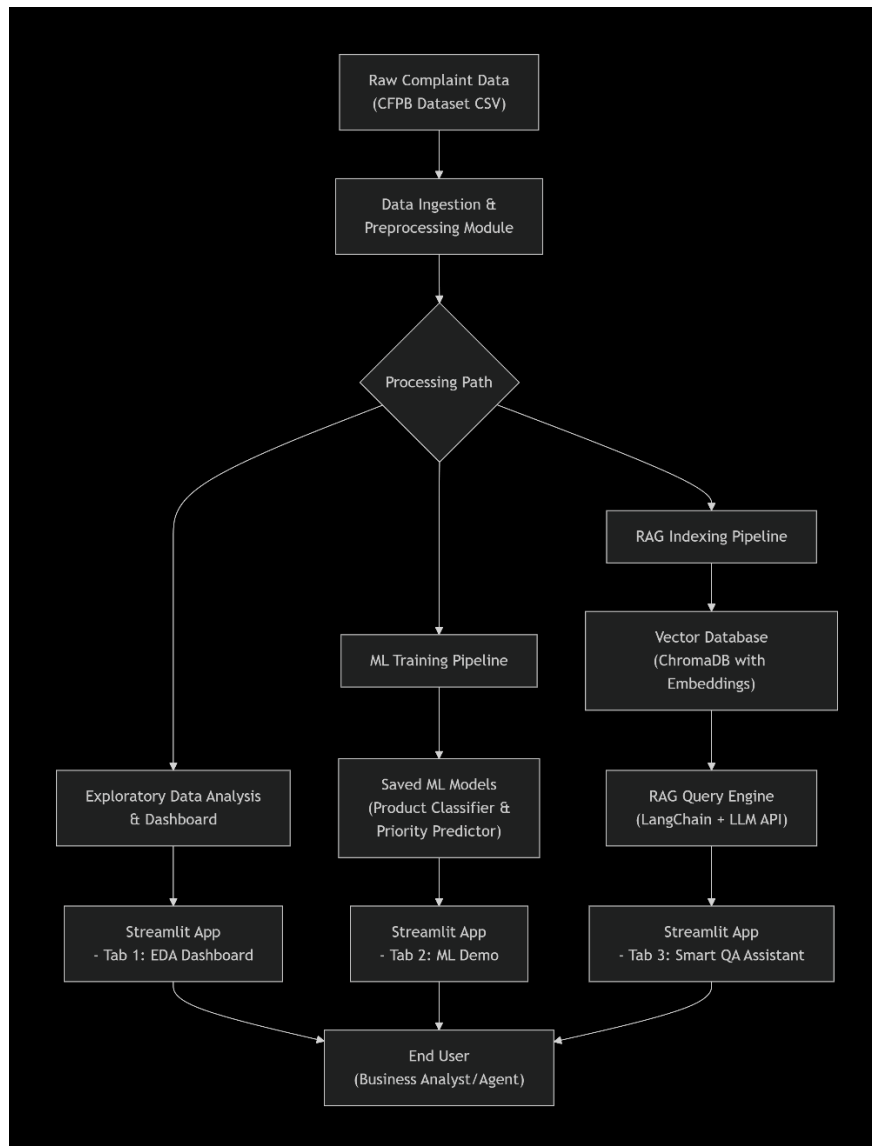
Nếu model không tương thích (ví dụ: dùng vocabulary khác), hệ thống báo lỗi chi tiết và từ chối upload.

A2: New model performs worse.

Nếu metrics của model mới thấp hơn model cũ một ngưỡng đáng kể (ví dụ: >5%), hệ thống hiển thị cảnh báo đỏ và yêu cầu người dùng xác nhận lại trước khi deploy.

IV. CÁCH HOẠT ĐỘNG (How It Works- System Architecture)

Hệ thống được thiết kế theo kiến trúc module, luồng xử lý chính như sau:



4.1. Module Thu thập và Tiền xử lý Dữ liệu:

- **Đầu vào:** File CSV gốc từ CFPB.

- **Xử lý:**

+ Đọc dữ liệu bằng Pandas.

- + Làm sạch: Xử lý missing values (loại bỏ hoặc điền giá trị), chuẩn hóa định dạng ngày tháng, loại bỏ ký tự đặc biệt trong text.
- + Lọc và chọn lựa các trường quan trọng: product, sub_product, complaint_narrative, company, date_received, timely_response.
- + Tạo nhãn cho bài toán ưu tiên: priority = "High" nếu timely_response == "No", ngược lại là "Low" (có thể mở rộng thành 3 lớp).
- + Lưu dữ liệu đã làm sạch thành file CSV/Parquet mới hoặc vào SQLite.

4.2. Module Phân tích và Dashboard (EDA):

- **Đầu vào:** Dữ liệu đã làm sạch.
- **Xử lý:**
 - + Sử dụng Pandas để tính toán các số liệu thống kê: số lượng khiếu nại theo sản phẩm, theo công ty, theo tháng.
 - + Sử dụng Plotly/Seaborn để tạo biểu đồ: bar chart, line chart (trend), heatmap, word cloud.
- **Đầu ra:** Một tab trong Streamlit App hiển thị các biểu đồ tương tác, có bộ lọc (dropdown, date picker) để người dùng tự khám phá.

4.3. Module Machine Learning (Predictive Modeling):

- **Bài toán A - Phân loại Sản phẩm (Product Classification):**
 - + **Đầu vào:** Cột complaint_narrative (text).
 - + **Nhãn:** Cột product (categorical).
 - + **Pipeline:** Text Data -> TF-IDF Vectorization -> Classifier (Logistic Regression / Random Forest / XGBoost). Huấn luyện, tinh chỉnh siêu tham số, đánh giá.
- **Bài toán B - Dự đoán Mức độ Ưu tiên (Priority Prediction):**
 - + **Đầu vào:** complaint_narrative + các features khác (có thể thêm độ dài text, sentiment score).
 - + **Nhãn:** Cột priority (binary: High/Low).
 - + **Pipeline:** Tương tự Bài toán A, nhưng là bài toán Binary Classification.
- **Output:** Hai file model được lưu (.pkl), một script predict.py để load model và dự đoán, và một giao diện trong Streamlit App để nhập văn bản và xem kết quả dự đoán.

4.4. Module LLM & RAG (Smart QA Assistant):

- **Giai đoạn Indexing:**

+ Chia toàn bộ văn bản trong complaint_narrative thành các đoạn nhỏ (chunks) có kích thước phù hợp (e.g., 500 ký tự).

+ Sử dụng Embedding Model (OpenAI text-embedding-3-small hoặc all-MiniLM-L6-v2) để chuyển mỗi chunk thành một vector số.

+ Lưu các vector này cùng với metadata (product, company, date) vào ChromaDB.

- Giai đoạn Querying:

+ Khi người dùng nhập câu hỏi tự nhiên (ví dụ: "Cho tôi tóm tắt các vấn đề về Bank of America trong tháng 1"):

- Câu hỏi được chuyển thành vector embedding (dùng cùng model trên).
- Thực hiện tìm kiếm similarity (cosine similarity) trong ChromaDB để tìm ra **k** chunk văn bản liên quan nhất.
- Kết hợp các chunk này thành "ngữ cảnh" (context).
- Tạo một prompt có cấu trúc gửi đến LLM (GPT-3.5-turbo), trong prompt chứa: *Hướng dẫn vai trò, Context tìm được, Câu hỏi của người dùng, Yêu cầu định dạng câu trả lời.*
- LLM sinh ra câu trả lời dựa trên ngữ cảnh được cung cấp, hạn chế "bịa đặt".

- **Đầu ra:** Một tab Chatbot trong Streamlit App, nơi người dùng có thể đặt câu hỏi và nhận câu trả lời thông minh, có căn cứ từ dữ liệu nội bộ.

V. CẤU TRÚC THƯ MỤC (Folder Tree)

Việc này giúp mọi người nắm được tổ chức code ngay từ đầu.

```
sccra-project/
|
├── data/                # Thư mục dữ liệu
|   ├── raw/             # Dữ liệu gốc (CFPB complaint data.csv)
|   ├── processed/       # Dữ liệu đã làm sạch (cleaned_data.csv)
|   └── database/        # File SQLite/ChromaDB
|
├── notebooks/          # Jupyter Notebooks cho thử nghiệm
|   ├── 01_eda_initial_analysis.ipynb
|   ├── 02_text_classification_experiment.ipynb
|   └── 03_rag_prototyping.ipynb
|
├── src/                # Mã nguồn chính (production-ready)
|   ├── data_pipeline/   # Xử lý dữ liệu
|   |   ├── __init__.py
|   |   ├── data_loader.py # Đọc và validate dữ liệu
|   |   ├── data_cleaner.py # Làm sạch và biến đổi
|   |   └── feature_engineer.py # Tạo features mới
|   |
|   └── ml_pipeline/     # ML models
```

```

| | | ├── __init__.py
| | | ├── train.py          # Script huấn luyện
| | | ├── predict.py       # Script dự đoán
| | | └── models/          # Thư mục lưu model
| | |     ├── product_classifier.pkl
| | |     └── priority_predictor.pkl
| |
| | ├── rag_pipeline/      # Hệ thống RAG
| | | ├── __init__.py
| | | ├── vector_store.py  # Tạo và quản lý vector DB
| | | ├── retriever.py     # Truy xuất thông tin
| | | └── qa_chain.py      # Chain với LLM
| |
| | └── app/               # Ứng dụng Streamlit
| | | ├── __init__.py
| | | ├── dashboard.py    # Tab 1: EDA Dashboard
| | | ├── ml_demo.py      # Tab 2: ML Demo
| | | ├── qa_assistant.py  # Tab 3: QA Assistant
| | | └── main.py         # App chính (tích hợp các tab)
| |
| | └── tests/             # Unit tests và integration tests
| | | ├── test_data_pipeline.py
| | | ├── test_ml_pipeline.py
| | | └── test_rag_pipeline.py
| |
| | └── config/            # File cấu hình
| | | ├── paths.yaml      # Đường dẫn đến file/data
| | | ├── model_params.yaml # Hyperparameters cho model
| | | └── api_keys.yaml    # API keys (được gitignore)
| |
| | └── docs/              # Tài liệu bổ sung
| | | ├── architecture_diagram.png # Sơ đồ kiến trúc
| | | ├── api_documentation.md     # API docs (nếu có)
| | | └── user_manual.md           # Hướng dẫn sử dụng app
| |
| | ├── requirements.txt    # Danh sách thư viện cần thiết
| | ├── environment.yml     # Conda environment
| | ├── .gitignore         # Git ignore file
| | ├── README.md          # Tài liệu chính của dự án
| | └── run.py             # Script khởi chạy toàn bộ pipeline

```

VI. XÂY DỰNG DỰ ÁN (Development Roadmap)

Giai đoạn 1: Thiết lập & Khám phá (Setup & Discovery) - 2 Tuần

1. Thiết lập môi trường ảo (venv/conda), cài đặt thư viện cốt lõi.

2. Tải dataset CFPB, import vào Pandas.
3. Thực hiện EDA cơ bản trong Notebook: hiểu schema, phân bố dữ liệu, tìm missing values.
4. Thiết kế cấu trúc thư mục cho dự án.
5. **Deliverable:** Một Notebook EDA và bản thiết kế hệ thống cơ bản.

Giai đoạn 2: Xây dựng Nền tảng & Xử lý Dữ liệu (Foundation & Data Pipeline) - 2 Tuần

1. Viết script `data_preprocessing.py` để tự động hóa việc làm sạch dữ liệu.
2. Thiết kế và tạo cấu trúc cơ sở dữ liệu (SQLite) cho dữ liệu đã làm sạch.
3. Xây dựng tab **EDA Dashboard** cơ bản trên Streamlit.
4. **Deliverable:** Pipeline xử lý dữ liệu chạy được và một app Streamlit với tab Dashboard.

Giai đoạn 3: Phát triển Mô hình ML (ML Model Development) - 3 Tuần

1. Phân chia dữ liệu cho bài toán Classification.
2. Thử nghiệm các phương pháp vector hóa văn bản (BoW, TF-IDF) và các mô hình khác nhau.
3. Huấn luyện, đánh giá và chọn lựa model tốt nhất cho từng bài toán.
4. Viết script `train.py` và `predict.py` cho model.
5. Tích hợp model vào Streamlit App (Tab 2).
6. **Deliverable:** Hai model được huấn luyện và một tab "ML Demo" hoạt động.

Giai đoạn 4: Triển khai Hệ thống RAG & LLM (RAG & LLM Integration) - 3 Tuần

1. Thiết lập ChromaDB và thử nghiệm embedding model.
2. Viết script `create_vector_db.py` để index toàn bộ dữ liệu vào vector database.
3. Học và áp dụng LangChain để xây dựng pipeline RAG: Retriever + LLM Chain.
4. Tích hợp RAG pipeline vào Streamlit App (Tab 3).
5. Tối ưu prompt engineering để cải thiện chất lượng câu trả lời.
6. **Deliverable:** Một vector database đầy đủ và một tab "Smart QA Assistant" có thể trả lời câu hỏi.

Giai đoạn 5: Hoàn thiện, Tối ưu & Triển khai (Polishing & Deployment) - 2 Tuần

1. Cải thiện giao diện người dùng (UI/UX) của Streamlit App.
2. Viết tài liệu đầy đủ (README.md, docstring).
3. Đóng gói code, kiểm tra lỗi.
4. Deploy ứng dụng lên Streamlit Community Cloud.
5. Chuẩn bị báo cáo tổng kết và demo.

6. **Deliverable:** Ứng dụng hoàn chỉnh được deploy public, repo GitHub chuyên nghiệp.

VII. THÁCH THỨC & GIẢI PHÁP (Technical Challenges & Solutions)

Phần này thể hiện **độ sâu tư duy** của dự án.

Thách thức	Giải pháp Đề xuất	Công cụ/Kỹ thuật Áp dụng
Dữ liệu văn bản không cân bằng (Imbalanced Text Data)	Sử dụng kỹ thuật resampling (SMOTE-NC cho text) hoặc class weights trong model.	imbalanced-learn, class_weight='balanced' trong scikit-learn
Tính toán Embedding cho lượng lớn văn bản	Sử dụng batch processing, caching embeddings, hoặc dùng model embedding nhẹ.	sentence-transformers, lưu embedding vào ChromaDB để tái sử dụng
Chi phí LLM API (OpenAI)	Implement caching cho câu hỏi tương tự, sử dụng model nhỏ hơn (gpt-3.5-turbo), set token limit.	Redis cache, prompt optimization, max_tokens parameter
Triển khai RAG có độ trễ cao	Optimize chunk size, sử dụng approximate nearest neighbor search, pre-compute embeddings.	FAISS với HNSW index, chunk size tuning (300-600 tokens)
Model drift theo thời gian	Thiết lập pipeline retraining định kỳ (monthly) với dữ liệu mới.	Apache Airflow (or simple cron job), model versioning với MLflow
Xử lý ngôn ngữ tự nhiên đa dạng (tiếng lóng, từ viết tắt)	Sử dụng spell correction, text normalization, và model embedding đã được pre-train trên dữ liệu đa dạng.	pyspellchecker, textblob, BERT-based embeddings

VIII. KẾ HOẠCH MỞ RỘNG (Future Roadmap & Extensions)

Phần này cho thấy dự án có **tầm nhìn dài hạn**.

Giai đoạn tiếp theo (Next Phase):

- Multi-language Support:** Thêm xử lý tiếng Việt (sử dụng PhoBERT cho embedding, viLLM cho LLM).
- Real-time Processing:** Kết nối với API nhận khiếu nại thời gian thực, xử lý streaming.
- Advanced Analytics:** Thêm model phát hiện sentiment, clustering chủ đề tự động (Topic Modeling với BERTopic).

4. **Automated Report Generation:** Tự động tạo báo cáo PDF/Email hàng tuần từ insights.

Giai đoạn nâng cao (Advanced Phase):

1. **Agentic Workflow:** Xây dựng agent AI có thể tự động tạo ticket, gửi email phản hồi mẫu dựa trên phân loại.
2. **Multimodal Input:** Xử lý khiếu nại có kèm ảnh (VLM để phân tích ảnh sản phẩm lỗi).
3. **Microservices Architecture:** Tách các module thành service độc lập (ML Service, RAG Service, Dashboard Service).
4. **A/B Testing Framework:** Đánh giá hiệu quả của các model/phương pháp khác nhau.

Các tích hợp tiềm năng (Potential Integrations):

- **CRM Systems:** Tích hợp với Salesforce, Zendesk để tự động cập nhật ticket.
- **Slack/Microsoft Teams Bot:** Đưa tính năng QA Assistant vào chat workspace.
- **Cloud Data Warehouse:** Kết nối với BigQuery/Snowflake để lấy dữ liệu trực tiếp.

IX. METRICS ĐÁNH GIÁ (Success Metrics)

Cách đo lường hiệu quả dự án một cách định lượng:

Metric	Công thức/Mô tả	Mục tiêu
ML Model Accuracy	(Số dự đoán đúng) / (Tổng số dự đoán)	> 85% cho Product Classification
Model Precision (cho High Priority)	(True High Priority) / (Tất cả dự đoán High)	> 80% (tránh báo động sai)
RAG Retrieval Hit Rate	(Số câu hỏi có context phù hợp) / (Tổng câu hỏi)	> 90%
End-to-end Latency	Thời gian từ lúc nhập khiếu nại đến lúc có kết quả phân loại	< 5 giây
User Adoption Rate	Số người dùng hoạt động / Tổng số người dùng được cấp quyền	> 70% trong tháng đầu
Time Saved per Complaint	Thời gian xử lý thủ công - Thời gian xử lý với hệ thống	> 2 phút/khiếu nại